



UFRJ



INSTITUTO DE
COMPUTAÇÃO
UFRJ

Programação de Computadores II

Laboratório - Métodos de Ordenação

Prof. Marcos Tomazzoli Leipnitz

Exercício 1

- Altere os algoritmos *bubble sort*, *insertion sort* e *selection sort*, vistos em aula, para que estes ordenem uma lista de inteiros em ordem decrescente ao invés de em ordem crescente.
- Crie um programa principal para testar as funções propostas.

Exercício 2

O algoritmo *bubble sort* ordena um vetor realizando várias passagens, trocando elementos adjacentes fora de ordem. Na versão tradicional, o número de comparações é sempre o mesmo, mesmo quando o vetor já está ordenado, resultando em complexidade $O(n^2)$ no melhor caso. No entanto, é possível melhorar o algoritmo para que, quando o vetor já estiver ordenado, o número de comparações seja proporcional a ao tamanho do vetor.

Exercício 2

Tarefa: Implemente a uma versão otimizada do *bubble sort*. Seu código deve manter a lógica original do algoritmo original, visto em aula, mas evitar comparações desnecessárias, de forma que a complexidade no melhor caso seja $O(n)$. Não utilize estruturas auxiliares nem bibliotecas de ordenação.

Exercício 3

- Escreva versões recursivas dos seguintes algoritmos vistos em aula: *bubble sort*, *insertion sort* e *selection sort*.
- Crie um programa principal para testar as funções propostas.

Exercício 4

- Critique os códigos das funções **check1** e **check2** a seguir, que prometem decidir se um array **v[0 . . n-1]** está em ordem crescente.

```
int check1 (int v[], int n) {  
    if (n > 1)  
        for (int i = 1; i < n; i++)  
            if (v[i-1] > v[i]) return 0;  
    return 1; }
```

```
int check2 (int v[], int n) {  
    int yes;  
    for (int i = 1; i < n; i++)  
        if (v[i-1] <= v[i]) yes = 1;  
        else {  
            yes = 0;  
            break; }  
    return yes; }
```

Exercício 5

- Considere as funções já definidas referentes às operações básicas sobre listas ligadas, vistas em aula, para resolução do exercício 5 a seguir.

Implementação da Lista Ligada

```
#include <stdio.h>
#include <stdlib.h>
#define true 1
#define false 0

typedef int bool;

typedef int TIPOCHAVE;

typedef struct {
    TIPOCHAVE chave;
    // outros campos...
} REGISTRO;

typedef struct aux {
    REGISTRO reg;
    struct aux* prox;
} ELEMENTO;

typedef ELEMENTO* PONT;

typedef struct {
    PONT inicio;
} LISTA;

void inicializarLista(LISTA* l){
    l->inicio = NULL;
}

int tamanho(LISTA* l) {
    PONT end = l->inicio;
    int tam = 0;
    while (end != NULL) {
        tam++;
        end = end->prox;
    }
    return tam; }
```

```
void exibirLista(LISTA* l) {
    PONT end = l->inicio;
    printf("Lista: \" \");
    while (end != NULL) {
        printf("%i ", end->reg.chave);
        end = end->prox;
    }
    printf("\n\n");
}

bool insere(LISTA* l, REGISTRO reg, int pos) {
    if (pos<0 || pos>tamanho(l)) return false;
    ELEMENTO* novo = (ELEMENTO*) malloc(sizeof(ELEMENTO));
    novo->reg = reg;
    int i;
    ELEMENTO* p;
    if (pos == 0){
        novo->prox = l->inicio;
        l->inicio = novo;
    }else{
        p = l->inicio;
        for (i=0;i<pos-1;i++) p = p->prox;
        novo->prox = p->prox;
        p->prox = novo;
    }
    return true;
}

bool exclui(LISTA* l, int pos) {
    if (pos<0 || pos>tamanho(l)-1) return false;
    int i;
    ELEMENTO* p;
    ELEMENTO* apagar;
    if (pos == 0) {
        apagar = l->inicio;
        l->inicio = apagar->prox;
    }else {
        p = l->inicio;
        for (i=0;i<pos-1;i++) p = p->prox;
        apagar = p->prox;
        p->prox = apagar->prox;
    }
    free(apagar);
    return true;
}

void reinicializarFila(LISTA* l) {
    PONT end = l->inicio;
    while (end != NULL) {
        PONT apagar = end;
        end = end->prox;
        free(apagar);
    }
    l->inicio = NULL;
}
```

Exercício 5

- Altere a implementação das funções **insere** e **exclui** para, respectivamente, inserirem e excluírem um determinado elemento de forma a manter os elementos armazenados na lista ligada de forma ordenada (em ordem crescente) pelo campo **chave** (considere que não há repetição do valor do campo **chave** - é único, identifica cada elemento). As demais funções devem continuar funcionando adequadamente, sem modificações.

Desafio 9

- Faça uma função que implemente o algoritmo *Insertion Sort* para ordenação dos dados da estrutura de dados **Tpessoa** a seguir, por ordem decrescente do valor do campo **nome**. Construa um programa principal para testar sua solução. Use a mesma formatação do exemplo de E/S a seguir. O primeiro valor indica a quantidade de pessoas com dados a serem informados.

```
typedef struct pessoa {  
    char nome[30];  
    int idade;  
} Tpessoa;
```

Desafio 9

- Exemplo de entrada (stdin) e saída (stdout)

Entrada	Saída
4	Jose: 35 anos
Joao 20	Joao: 20 anos
Jose 35	Carlos: 21 anos
Ana 30	Ana: 30 anos
Carlos 21	